

Documentação do Projeto

Análise de Desempenho de Algoritmos de Ordenação (BubbleSort, InsertionSort e ShellSort)

1. Enunciado do Problema

Objetivo: Implementar os seguintes métodos de ordenação: BubbleSort; InsertionSort; ShellSort;

Requisitos de Dados: Criar dois conjuntos de vetores de dados (dataset) com valores aleatórios, sendo que o tamanho de cada vetor é dado a seguir de acordo com os dois conjuntos:

- **Dataset 1:** 10 – 20 – 30 – 40 – 50 – 60 – 70 – 80 – 90 - 100
- **Dataset 2:** 1000 - 2000 – 3000 – 4000 – 5000 – 6000 – 7000 – 8000 – 9000 - 10000

Regras de Execução: Cada método de ordenação deve receber os 20 vetores como entrada (receber sempre os mesmos vetores) e devolver o vetor ordenado. Para cada execução tomar o tempo de execução (calcular a média de 100 execuções).

Requisitos de Estrutura: O projeto deverá conter no mínimo 4 classes:

- **Classe Ordena:** contém as implementações dos algoritmos de ordenação;
- **Classe Item:** contém o objeto a ser ordenado;
- **Classe Utilidades:** contém a geração dos vetores, leitura e escrita de arquivos e demais utilidades;
- **Classe TesteOrdena:** realiza a tomada de tempo.

2. Visão Geral

Este projeto consiste na implementação e análise comparativa de três algoritmos clássicos de ordenação da ciência da computação: **BubbleSort**, **InsertionSort** e **ShellSort**. O objetivo principal é avaliar a eficiência empírica de cada algoritmo quando submetidos a diferentes volumes de dados empacotados em objetos complexos, medindo o tempo médio de processamento ao longo de múltiplas execuções contínuas.

3. Estrutura de Classes (Arquitetura)

O sistema foi desenvolvido utilizando o paradigma de Orientação a Objetos na linguagem Java, estruturado em quatro classes principais para garantir uma clara separação de responsabilidades:

Classe	Responsabilidade	Principais Métodos / Atributos
Item	Representa a unidade básica de dados que será ordenada dentro dos arranjos, encapsulando a chave de ordenação.	Atributo: chave (int) Métodos: getChave(), setChave()
Ordena	Funciona como uma biblioteca de algoritmos, contendo as lógicas puras de ordenação de forma estática.	bubbleSort(Item[] vetor) insertionSort(Item[] vetor) shellSort(Item[] vetor)
Utilidades	Concentra rotinas de apoio, como a geração automatizada de vetores com valores aleatórios.	gerarVetorAleatorio(int tamanho)
TesteOrdena	Ponto de entrada do sistema (main). Coordena a geração dos conjuntos de dados, o controle rigoroso de tempo e o cálculo das médias.	main(String[] args)

4. Algoritmos Implementados e Complexidade

4.1 BubbleSort

Percorre o vetor repetidamente, comparando elementos adjacentes e trocando-os de posição se estiverem na ordem errada. O maior elemento "flutua" para o final a cada iteração completa. Possui complexidade de tempo de $O(n^2)$ no caso médio e pior caso.

4.2 InsertionSort

Constrói a matriz ordenada final um elemento por vez, consumindo um elemento de entrada a cada iteração e inserindo-o na sua posição correta dentro da fração já ordenada do vetor. Embora possua complexidade de pior caso de $O(n^2)$, é extremamente eficiente para pequenos volumes de dados.

4.3 ShellSort

Uma extensão refinada do InsertionSort que permite a troca de elementos distantes. O vetor é segmentado em subvetores baseados em um espaçamento (*gap*) que decresce progressivamente até atingir 1. Possui complexidade típica entre $O(n \log^2 n)$ e $O(n^{1.5})$, dependendo da sequência de gaps.

5. Metodologia de Testes e Geração de Dados

Para assegurar a integridade científica e evitar distorções estatísticas, adotou-se a seguinte metodologia:

- **Isolamento de Variáveis:** Para cada tamanho de vetor, um conjunto único de dados aleatórios foi gerado. Os três algoritmos foram testados exatamente com a mesma entrada original.

- **Prevenção contra Pré-Ordenação (Clonagem):** Como os testes exigem 100 repetições por tamanho para o cálculo da média, uma cópia idêntica do vetor original (via `Arrays.copyOf()`) foi fornecida ao algoritmo antes de disparar o cronômetro. Isso garante que o algoritmo nunca trabalhe com dados pré-ordenados pelas execuções anteriores.
- **Alta Precisão Cronométrica:** A medição de tempo utilizou o relógio de alta resolução do sistema através do método `System.nanoTime()`, convertendo a média final acumulada para milissegundos (*ms*).

6. Resultados de Desempenho Obtidos

Abaixo está consolidada a tabela de tempos médios obtidos após as 100 execuções para cada tamanho de vetor (Datasets 1 e 2):

Tamanho do Vetor	BubbleSort (ms)	InsertionSort (ms)	ShellSort (ms)
10	0,011185	0,001610	0,001998
20	0,005110	0,003740	0,002070
30	0,002618	0,001366	0,000931
40	0,006058	0,003646	0,002306
50	0,013997	0,003980	0,002049
60	0,009728	0,008448	0,002566
70	0,004447	0,038993	0,005550
80	0,004884	0,002136	0,004985
90	0,006382	0,002619	0,001361
100	0,010926	0,003914	0,002414
1.000	0,793748	0,203053	0,070994
2.000	3,506588	0,885744	0,178863
3.000	6,862457	1,724635	0,266828
4.000	11,931642	2,913918	0,362270
5.000	18,898054	4,541219	0,459817
6.000	27,363829	6,578933	0,601354
7.000	43,346723	9,461490	0,729153
8.000	54,037963	12,206237	0,896241
9.000	68,382651	14,524355	0,929610
10.000	91,693662	19,412847	1,185655

7. Conclusões Experimentais

Crescimento Quadrático do BubbleSort: Conforme previsto pela análise assintótica, o BubbleSort demonstrou um aumento severo de tempo à medida que o tamanho do vetor escalou. Para 10.000 elementos, atingiu expressivos **91,69 ms**, evidenciando o alto custo de suas trocas sucessivas em memória.

Vantagem Prática do InsertionSort: Apesar de compartilhar do mesmo limite superior teórico do BubbleSort ($O(n^2)$), o InsertionSort completou o maior vetor em apenas **19,41 ms** (cerca de 4,7 vezes mais veloz). Isso decorre de seu menor número de movimentações e melhor aproveitamento de subarrays parcialmente ordenados.

Supremacia Absoluta do ShellSort: O ShellSort exibiu um comportamento de alta performance e estabilidade impecável. Ao saltar de 1.000 para 10.000 itens, seu tempo variou de meros 0,07 ms para apenas **1,18 ms**. A quebra de barreiras de posições distantes por meio dos múltiplos gaps provou ser a estratégia mais otimizada para conjuntos de dados de larga escala.

